

1) Selection Sort

1. Initialize i with 0
2. Find the smallest item x , in the range of $[1 \dots n-1]$
3. Swap x with the i^{th} item
4. Increase i by 1 & goto step 2

2) Heap Sort

Heapify - $\log n$
Combined - $n \log n$

- The heap sort is an improved version of the selection sort in which the largest element (the root) is selected and exchanged with the last element in the unsorted list.

Algo :-

Heap sort does this with the following steps-

1. Build a heap tree with the given set of data
2. a) Remove the top most item (the largest) & replace it with the last element in the heap
b) Re-heapify the complete binary tree.
c) Place the deleted node in the output
3. Continue step 2 until the heap tree is empty.

3. Insertion Sort (Back Traversing)

- Best $\rightarrow O(n)$
 - Worst $\rightarrow O(n^2)$
1. If the element is the first element, assume that it is already sorted. Return 1
 2. Pick the next element, and store it separately in a key.
 3. Now compare the key with all elements in the sorted array
 4. If the element in the sorted array is smaller than the current element, then move to the next element. Else, shift greater elements in the array towards the right
 5. Insert the value

4. Shell Sort (Unstable)

1. ~~a is given array, n is the size of the array~~
2. ~~Divide the array in 2 equal halves~~
~~for (interval = $n/2$; interval > 0; interval /= 2)~~
 1. Start
 2. Initialize the value of gap size, h
 3. Divide the list into smaller sub-part. Each must have equal intervals to h
 4. Sort these sub-lists using insertion sort
 5. Repeat this step 2 until the list is sorted
 6. Print a sorted list
 7. Stop

5. Bubble Sort

Best case - $O(n)$
Avg Case - $O(n^2)$
Worst Case - $O(n^2)$

Space $\rightarrow O(1)$
STABLE

1. Initialize i with 0
2. Initialize j with $i+1$
3. Compare $a[i]$ and $a[j]$
4. Swap $a[i]$ and $a[j]$ if $a[i] > a[j]$
5. Increment i by 1 and goto step 4.

6. Quick Sort

Avg $\rightarrow O(n \log n)$

QuickSort (array A, start, end)

1. if (start < end)
2. $\{$
3. $p = \text{partition}(A, \text{start}, \text{end})$
4. QuickSort (A, start, $p-1$)
5. QuickSort (A, $p+1$, end)
6. $\}$
7. $\}$

7. Merge Sort

mergesort(arr, beg, end)

if (beg < end)

set mid = (beg + end) / 2

Mergesort(arr, beg, mid)

mergesort(arr, mid+1, end)

mergesort(arr, beg, mid, end)

end of if

End mergesort

8. Sequential Search

Linear Search (Slow)

1. Initialize an index variable i
2. Compare a[i] with target
 - If a[i] == target, found
 - If a[i] != target,
 - If all have checked already, not found
 - Otherwise, change i into next index and goto step 2

9. Binary Search

1. Initialize boundaries L and R
2. While L is still on the left of R
 - mid = (L + R) / 2
 - If a[mid] == Target, found
 - If a[mid] > target, set R be mid-1 and goto step 2
 - If a[mid] < target, set L be mid+1 and goto step 2

10. Hashing

Division Method $h(K) = K \text{ (modulo) Size}$
 $= K \% \text{ Size}$

Meet Square method $h(K) = K^2$

10, 15, 12
100, 225, 144
10, 15, 12
100, 225, 144

Folding method

$$h(K) = k_1 + k_2 + k_3 + \dots + k_n$$

10, 21, 11

1+0, 2+1, 1+1

1, 3, 2 → Index

Load factor = $\frac{\text{Keys}}{\text{Size}}$

Collision resolution techniques

Chaining - $(2K+3) \% \text{Size}$

Linear Probe - Iterations for empty space
= Probe

$$(h(K) + i) \% 10$$

Quadratic Probe - $h(K) = K \text{ mod } 10$
 $h'(K, i) = (h(K) + i^2) \text{ mod } 10$

Double Hashing - $h_1(K) = K \text{ mod } 11$
 $h_2(K) = 8 - (K \text{ mod } 8)$
 $[h_1(K) + i h_2(K)] \text{ mod } 11$

11. Disjoint Set

- Intersection → NULL

Union Find Algo

Union-find algo performs two very useful op -

Find: To find the subset a particular element 'K' belongs to. It is generally used to check

Types of Graphs

- Sparsely connected graphs
- Densely

$$\begin{matrix} V & E \\ n & \frac{1}{2}(n^2 - n) \end{matrix}$$

- Complete - Max no of edges
Dir $n(n-1)$
Undir $\frac{n(n-1)}{2}$

Spanning tree - All vertices
subgraph of G

Kruskal's Algo (Consider edges)

1. Sort all the arcs into ascending order of weight.
2. Select the arc of least weight to start the tree
3. Consider the next arc of least weight
 - If it would form a cycle with the arcs already selected reject it
 - If it does not form a cycle, add it to the tree.
 - If there is a choice of equal arcs then consider each in turn.
4. Repeat Step 3 until all the vertices are connected

Prim's Algorithm (Vertices)

1. Choose any vertex to start the tree.
2. Selection arc of least weight that joins a vertex that is already in the tree to a vertex that is not yet in the tree
If there is a choice of arcs of equal weight, choose randomly
3. Repeat step 2 until all the vertices are connected

Dijkstra's Algo (Shortest path)

1. Label the start vertex, S with the final label, 0
2. Record a working value at every vertex, V , which is directly connected to the vertex, X , which has just received its final label.
- Working value at $V = \text{Final value at } X + \text{weight of arc } XY$
- If there is already a working value at V , it is only replaced if the new value is smaller.
- Once a vertex has a final label it is not revisited & its working values are no longer considered

3. Look at the working values of all vertices without final labels. Select the smallest working value. This now becomes the final label at that vertex (If two vertices have the same smallest working value either may be given its final label first).
4. Repeat 2-3 until the destination vertex, T receives its final label
5. To find the shortest path, trace back from T to S . Given that B already lies on the route, include arc AB whenever final label of $B - \text{final label of } A = \text{weight of arc } AB$.

BFS

A standard BFS implementation puts each vertex of the graph into one of two categories:

- visited
- Not visited



BFS

1. Start by putting any one of the graph's vertices at the back of a queue
2. Take the front item of the queue & add it to the visited list.
3. Create a list of that vertex's adjacent nodes. Add the ones which aren't in the visited list to the back of the queue
4. Keep repeating steps 2 & 3 until the queue is empty

DFS

1. First, create a stack with the total number of vertices in the graph
2. Now choose any vertex as the starting point of traversal and push that vertex into the stack.
3. After that, push a non-visited vertex (adjacent to the vertex on the top of the stack) to the top of the stack.
4. Now repeat steps 3 & 4 until no vertices are left to visit from the vertex on the stack's top.
5. If no vertex is left, go back and pop a vertex from the stack
6. Repeat steps 2, 3 and 4 until the stack is empty